

ACCUTRACK

ID-Persistent Retail Analytics Platform

A technical overview of the implementation, machine learning pipelines, real-world engineering challenges faced, and resolutions implemented in the AcuTrack multi-camera tracking system.

Project Status: Active Development / Production Prototype

Technologies: YOLOv8 | StrongSort | OSNet | LSTM Autoencoder | Flask

Document Version: 1.0 (August 2026)

1. Executive Summary & Introduction

AccuTrack is an AI-powered retail analytics and crowd intelligence platform. In modern retail environments, understanding customer movement - such as entry rates (footfall), average dwell times in specific zones, and behavior trajectories - is critical to improving store layouts, optimizing staff distribution, and ensuring security.

The system processes multiple video streams concurrently, performing real-time person detection, multi-object tracking, cross-camera identity matching (re-identification), and trajectory anomaly detection. The results are broadcast via SocketIO to a web-based dashboard that provides real-time occupancy statistics, alerts, and historical statistics.

Core System Features:

- * **Real-time Multi-stream Tracking:** Processes 7 Wildtrack camera feeds simultaneously.
- * **Zone Analytics:** Calculates footfall counts, current occupancies, and average dwell times for zones.
- * **Cross-Camera Re-ID:** Maintains consistent identity tracking as users move between camera fields of view.
- * **Behavior Anomaly Detection:** Uses deep learning to score trajectories and flag unusual movements (loitering).

2. The Machine Learning Pipeline

AccuTrack achieves tracking and behavioral analysis by cascading four core machine learning and computer vision modules:

- * **YOLOv8 (Object Detection):** Detects persons in each frame using a YOLOv8 nano model (optimized as ONNX for CPU). It outputs bounding boxes around individuals with confidence scores.
- * **StrongSort (Multi-Object Tracking):** Maintains frame-to-frame associations. It links detections over consecutive frames within a single camera view, predicting movements using a Kalman filter and generating local track IDs.
- * **OSNet (Omni-Scale Re-ID Network):** Extracts a high-dimensional (512-float) feature vector (embedding) representing a person's visual appearance. This embedding is used to associate local tracks from different cameras to a unified Global ID.
- * **LSTM Autoencoder (Anomaly Detection):** Analyzes visitor trajectory sequences. Normal movement patterns have low reconstruction errors when passed through the trained encoder-decoder network. High reconstruction error indicates irregular movement (e.g. loitering, trespassing, or panic).

3. Detailed Logic & Algorithms

1. Dwell Time and Footfall Calculation

Every camera stream defines specific rectangular areas corresponding to retail zones (e.g. Entrance, Food Court, Exit Corridor). For each active track, the system calculates the center point of its bounding box and evaluates if it lies within a zone's coordinates:

- Entry: When a track enters a zone for the first time, the entry timestamp is recorded.
- Dwell time: The running difference between the current frame timestamp and the entry timestamp.
- Exit & Footfall: When a track exits the zone or disappears, the final dwell time is computed and added to a running averages list. The zone's historical footfall counter is incremented.

2. Cross-Camera Re-ID Gallery Logic

To track a customer across the entire store layout, we maintain a global gallery of known customer embeddings. When a new local track starts on any camera:

1. OSNet extracts its feature embedding.
2. The embedding is compared against all registered identities in the global gallery using cosine distance.
3. If the minimum distance is below a matching threshold (0.45) and is unambiguous (meaning it does not closely match multiple candidates), the local track is merged with the existing Global ID.
4. Otherwise, a brand new Global ID is created and added to the gallery.

3. LSTM Autoencoder Anomaly Score

The LSTM Autoencoder monitors spatial trajectories. It processes coordinates as relative displacements over a sliding window of 30 frames. The encoder compresses the sequence, and the decoder attempts to recreate it. The reconstruction loss is calculated as Mean Squared Error (MSE):

$$\begin{aligned}\text{Reconstruction Loss (MSE)} &= \text{Mean}((X_{\text{original}} - X_{\text{reconstructed}})^2) \\ \text{Anomaly Score} &= \text{ML_Weight} * \text{MSE} + \text{Rule_Weight} * (\text{Dwell_Time} / \text{Loiter_Limit})\end{aligned}$$

If the combined anomaly score exceeds the anomaly threshold, a real-time warning is triggered in the GUI.

4. Engineering Challenges & Resolutions

Deploying multiple deep learning models in real-time across 7 camera streams revealed several edge cases. Below are the key problems faced and how we resolved them:

PROBLEM CHANNELS & SYMPTOM:
Cross-Camera Global ID Collisions
Initially, when a new track was registered but its embedding wasn't ready yet, the tracker fell back to the local raw track_id. Because StrongSort on Camera 1 and Camera 2 both number tracks starting from 1, two different people ended up sharing the same Global ID=1, resulting in merged target locks and blended gallery statistics.

RESOLUTION

Namespaced Local ID Fallbacks
We introduced camera-namespaced negative integers as temporary fallback IDs (e.g., fallback ID = -(camera_index * 1000 + local_track_id)). Since all registered global gallery IDs are positive integers starting from 1, these negative fallbacks can never collide with each other or existing gallery IDs.

PROBLEM CHANNELS & SYMPTOM:
Embedding Degradation (Blurry Templates)
The gallery previously maintained a running average of matching embeddings for each Global ID. However, averaging embeddings taken from different angles (e.g., front-view vs top-view) produced a 'blurry' average vector that didn't match any viewpoint well, causing the system to lose track of customers when they changed angles.

RESOLUTION

Multi-Template Gallery Matching
Instead of keeping a single average embedding, the gallery now stores up to 6 distinct templates (historical captures) per person. Comparisons are made against the best-matching (minimum distance) template. This allows robust matching across multiple distinct viewing angles.

PROBLEM CHANNELS & SYMPTOM:
Thread Lock Contention & Frame Stutter
Originally, a single global YOLO lock and behavior lock was shared across all camera threads. This meant camera threads spent most of their time waiting for locks, dropping frames, and falling behind real-time speed.

RESOLUTION

Lock Splitting and Dedicated Model Instances
We modified the engine to run independent YOLO models and separate locks for each camera thread. Additionally, we introduced on-demand thread initialization - camera streams only start executing and consuming resources when the operator first views them.

PROBLEM CHANNELS & SYMPTOM:

Ghost Tracks and ID Fragmentation

With a low detection confidence threshold (0.20) and high tracking max_age (60 frames), the system suffered from ghost tracks (random shapes registered as stationary people) and ID fragmentation (a person walking out of view and returning registered as a new ID).

Threshold Tuning

We tuned the YOLO confidence threshold to 0.35 to eliminate false detections. The tracking max_age was tuned to 20/30 frames (about 4 to 6 seconds), which eliminates ghost track persistence while keeping temporarily occluded targets linked.

5. Database Schema Design

To enable historical analysis, footfall tracking, and alert logging, we transitioned the application from in-memory arrays to a SQL-based database using SQLAlchemy. Below is the mapping of the core schema tables:

- * **cameras:** Registers configured streams, URLs, status, and metadata.
- * **zones:** Defines the specific retail monitoring locations (coordinates, capacities, colors).
- * **tracked_persons:** Stores all unique re-identified visitors, when they were seen, and suspicious status.
- * **person_embeddings:** Stores raw high-dimensional embeddings mapping back to unique users for Re-ID.
- * **dwelt_times:** Logs visit intervals (entry, exit, duration) for each person per zone.
- * **tracked_persons_history:** Stores high-resolution coordinates for heatmaps and model re-training.
- * **occupancy_snapshots:** Records zone/camera occupancies periodically for dashboard chart loading.
- * **alerts:** Persists security and crowd limit violations for auditor lookup.

Verification & Future Improvements

By writing these statistics to a database, the frontend dashboard can load historical trends (e.g., peak footfall hours, zone popularity indices, average dwell times by week day). This lays the groundwork for cloud deployments, advanced customer behavior analytics dashboards, and multi-store analytics integration.